

Introduction to DL4MSc – Convolutional Neural networks

Prof. Alexander Binder

August 18, 2025

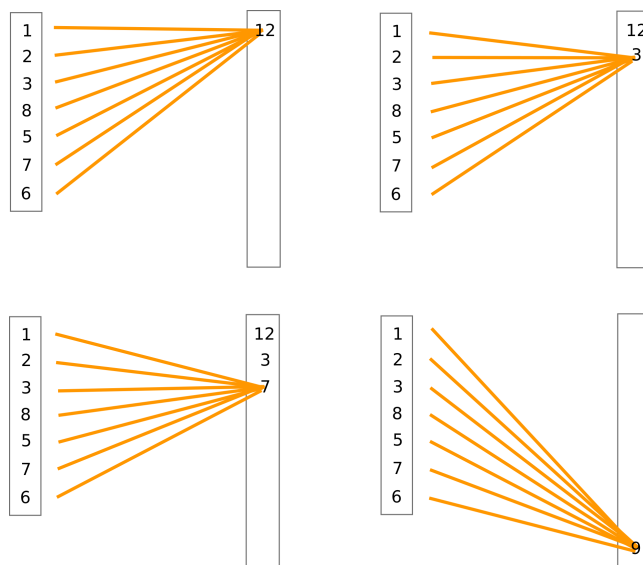
extra material:

https://d2l.ai/chapter_convolutional-neural-networks/index.html

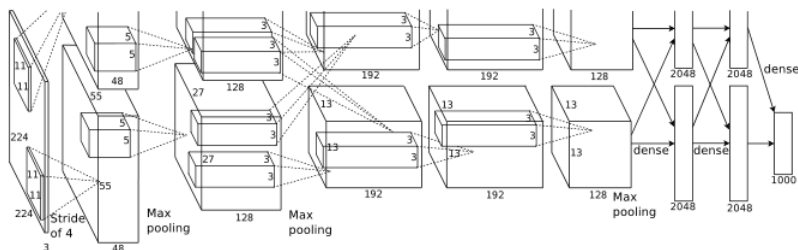
Key takeaways:

- Convolution in deep learning is essentially a sliding window of inner products. The inner product acts as a pattern detector.
- The pattern detectors in convolutions are locally applied within one position of the sliding window
- Advantages: Convolutions apply the same pattern detector in sliding windows over 1d-,2d-,nd-spatial dimensions
- Advantages: convolutions have less parameters than fully connected networks – easier to learn
- Advantages: stacking conv layers means to stack pattern detectors – higher layers detect more complex patterns built from simple ones
- Formulas: convolution with C input and D output channels
- Formulas: the effect of kernel size, stride and padding (=parameters of the convolution) on the output shape
- Formulas: be able to compute the size of a feature map (spatial dimensions and the number of output channels), when a convolution is applied
- Formulas: the theoretical receptive field size
- be able to explain how sum-pooling, max-pooling works
- be able to explain how dilated convolution works

1 Not a convolution: recap of the fully connected layer, 1 input channel



- Observation for fully connected nets: number of weights grows with the number of elements in the input and the output



taken from: Alex Krizhevsky et al., NIPS2012 <http://www.cs.toronto.edu/~fritz/absps/imagenet.pdf>

- how to connect neurons with less parameters when stacking layers?
 - **key idea:** in images neighbor pixels tend to be related! So we connect only neighboring neurons in the input.

2 Convolutional Neural networks

See also for example: <http://neuralnetworksanddeeplearning.com/chap6.html>.

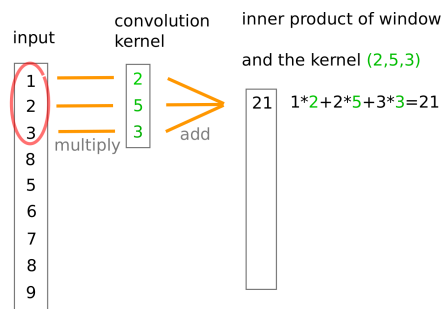
2.1 1-d convolutions, 1 input channel

Convolution layers compute inner products between an input tensor and a kernel tensor w along a sliding window moving along the input.

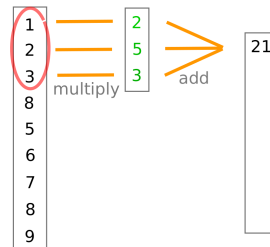
Convolution layers / Faltungsschichten in neuronalen Netzen berechnen Skalarprodukte zwischen der Faltungsmatrix (= Faltungskern) und einem Abschnitt des Eingabetensors, wobei der Abschnitt des Eingabetensors entsprechend des Wertes des stride-Parameters im Sinne eines beweglichen Fensters verschoben wird.

Für jeden Abschnitt erhält man ein Skalarprodukt, welches die Ähnlichkeit zwischen diesem Abschnitt des Eingabetensors und der Faltungsmatrix berechnet wird.

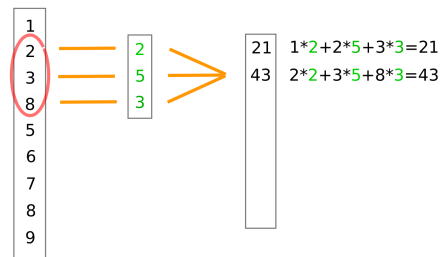
See the next graphic / siehe naechstes Bild.



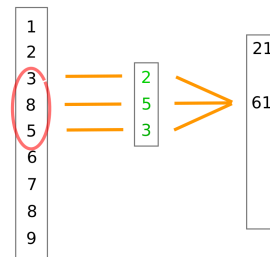
stride 1 = move kernel by 1 element



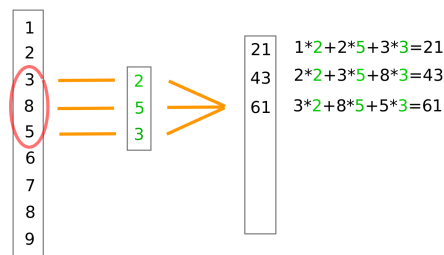
stride 2 = move kernel by 2 elements



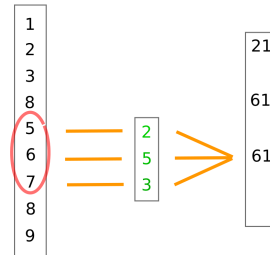
stride 1 = move kernel by 1 element



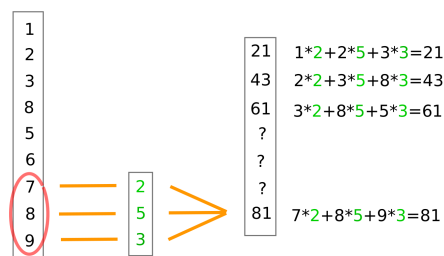
stride 2 = move kernel by 2 elements



stride 1 = move kernel by 1 element



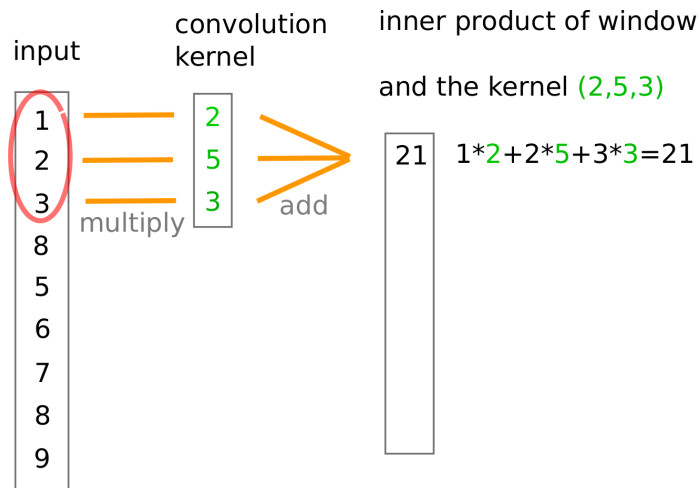
stride 2 = move kernel by 2 elements



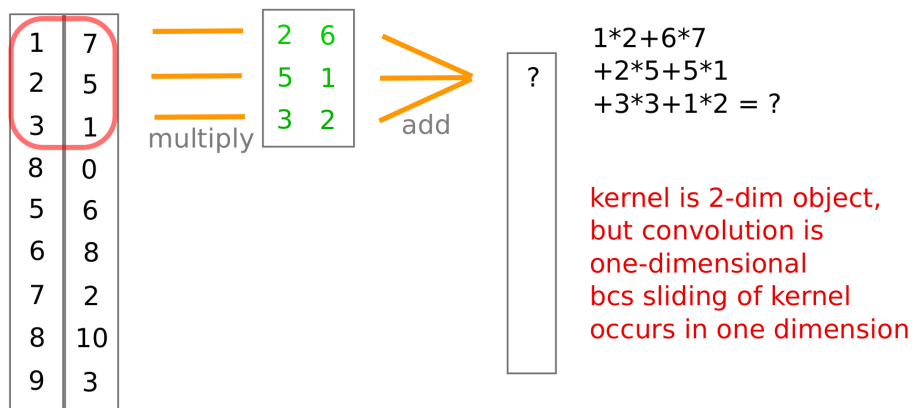
no padding of input:
 $\text{outputsize} = \text{inputsize} - 2 = \text{inputsize} - (\text{kernelsize} - 1)$

no padding of input:
 $\text{outputsize} = \text{ceil}((\text{inputsize} - 2) / 2)$
 $= \text{ceil}((\text{inputsize} - (\text{kernelsize} - 1)) / \text{stride})$

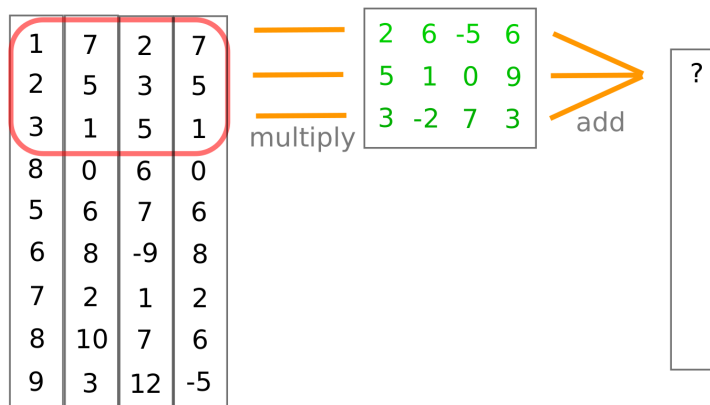
2.2 1-d convolutions, 2+ input channels



2 input channels, kernel is of size (nchannels, kernel size) = (2,3)



4 input channels, kernel is of size (nchannels, kernel size) = (4,3)

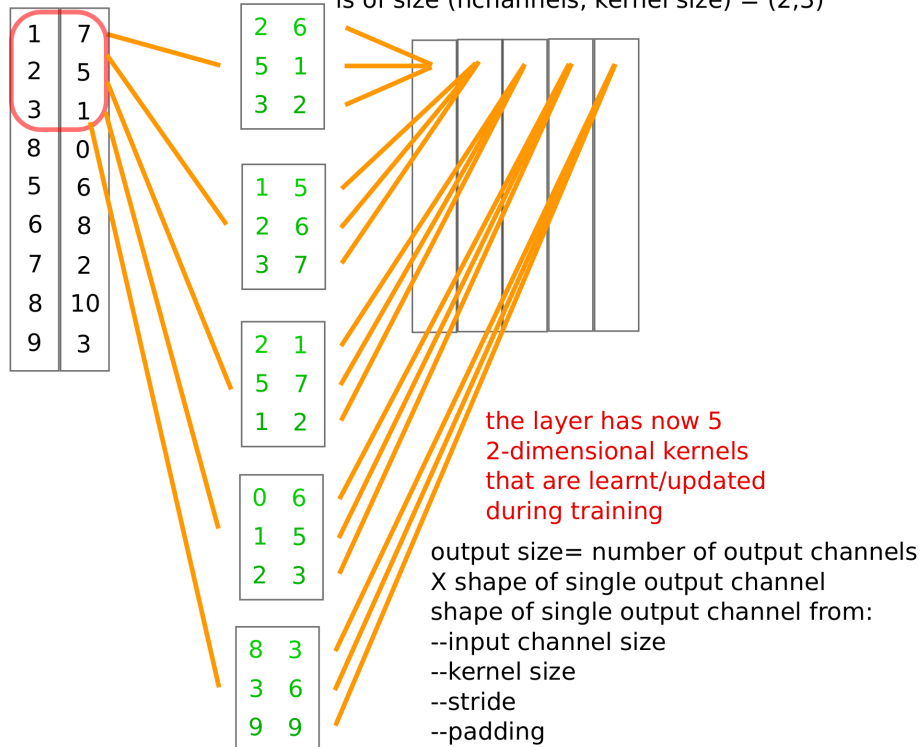


2.3 1-d convolutions, one whole convolution layer (multiple output channels)

5 output channels -- by 5 independent kernels

2 input channels, each of the five kernels

is of size (nchannels, kernel size) = (2,3)



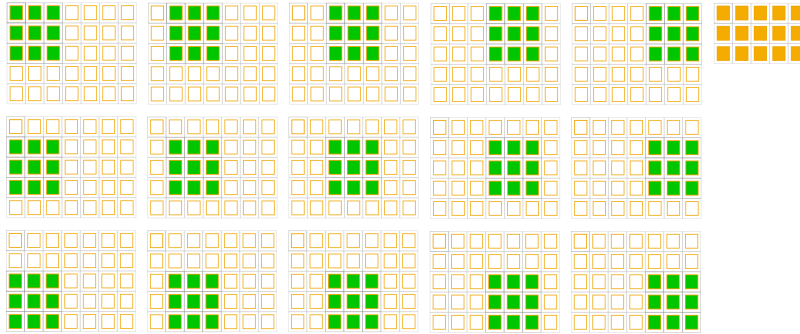
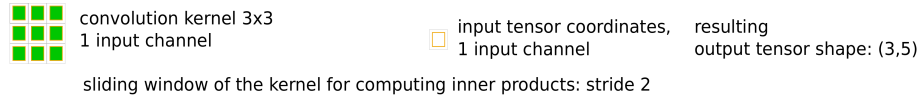
2.4 why use convolutions, argument I: – less parameters

Most neural net example code for mnist have linear (fully connected) layers. In it: each neuron of layer l is connected to each neuron of layer $l + 1$.

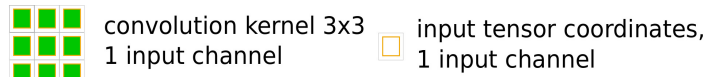
- Fully connected: Assume we have $N(l)$ neurons in layer l and $N(l + 1)$ neurons in layer $l + 1$: parameters have dimensionality =?
- Locally connected: Assume we have $N(l)$ neurons in layer l and $N(l + 1)$ neurons in layer $l + 1$, each output neuron takes input from a patch of 5 neighbors: parameters have dimensionality ?
- 1-d convolution with kernel size 5: parameters have dimensionality 5, no matter how many inputs or outputs in the sliding dimension
- convolutions have a small parameter dimensionality, independent of input and output size in the sliding dimension (number of output channels matters though)

2.5 2-d convolutions, 1 input channel

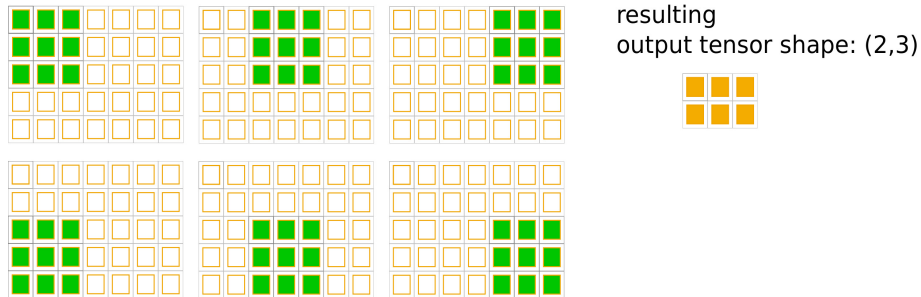
A simplified convolution (one input channel only) can be drawn like this for stride 1



A simplified convolution (one input channel only) can be drawn like this for stride 2



sliding window of the kernel for computing inner products: stride 2



In terms of math language a 2d-convolution in deep learning **for one input channel**, with stride 1, no padding (and dilation 1) can be expressed like this:

$$z[h, w] = \sum_{i_h=0}^{k_h-1} \sum_{i_w=0}^{k_w-1} u[i_h, i_w] x[h + i_h, w + i_w]$$

In image processing this is known as cross-correlation, however.

Image processing has its own definition of convolution, which would be:

$$\begin{aligned} z[h, w] &= \sum_{i_h=0}^{k_h-1} \sum_{i_w=0}^{k_w-1} u[i_h, i_w] x[h + k_h - 1 - i_h, w + k_w - 1 - i_w] \\ &= \sum_{i_h=0}^{k_h-1} \sum_{i_w=0}^{k_w-1} u[k_h - 1 - i_h, k_w - 1 - i_w] x[h + i_h, w + i_w] \end{aligned}$$

In Image processing the filter $\mathbf{u}$ is traversed in the opposite direction in each axis. A mirroring along the horizontal and the vertical axis is the same as rotating the filter by 180 degrees. Example to show this:

$$\begin{bmatrix} 1 & 2 \\ 4 & 3 \end{bmatrix} \xrightarrow{\text{mirror horizontal}} \begin{bmatrix} 2 & 1 \\ 3 & 4 \end{bmatrix} \xrightarrow{\text{mirror vertical}} \begin{bmatrix} 3 & 4 \\ 2 & 1 \end{bmatrix}$$

Take note of this point of confusion: convolution in deep learning is cross-correlation in image processing!!

2D-Convolution in image processing amounts to using a 180 degrees rotated filter compared to 2D-Convolution in deep learning

Technically, convolution as in deep learning is an inner product (no matter the kernel or feature map is traversed forward or backward):

$$\begin{aligned} z[h, w] &= \sum_{i_h=0}^{k_h-1} \sum_{i_w=0}^{k_w-1} u[i_h, i_w] x[h + i_h, w + i_w] \\ &= u \cdot x[h : h + k_h, w : w + k_w] \end{aligned}$$

Above shows one output element $z[h, w]$ at position (h, w) in the output feature map between u and the slice $x[h : h + k_h, w : w + k_w]$ of tensor x

Take note of the fact that convolution is a series of inner products between the filter u and a window-view of the tensor.

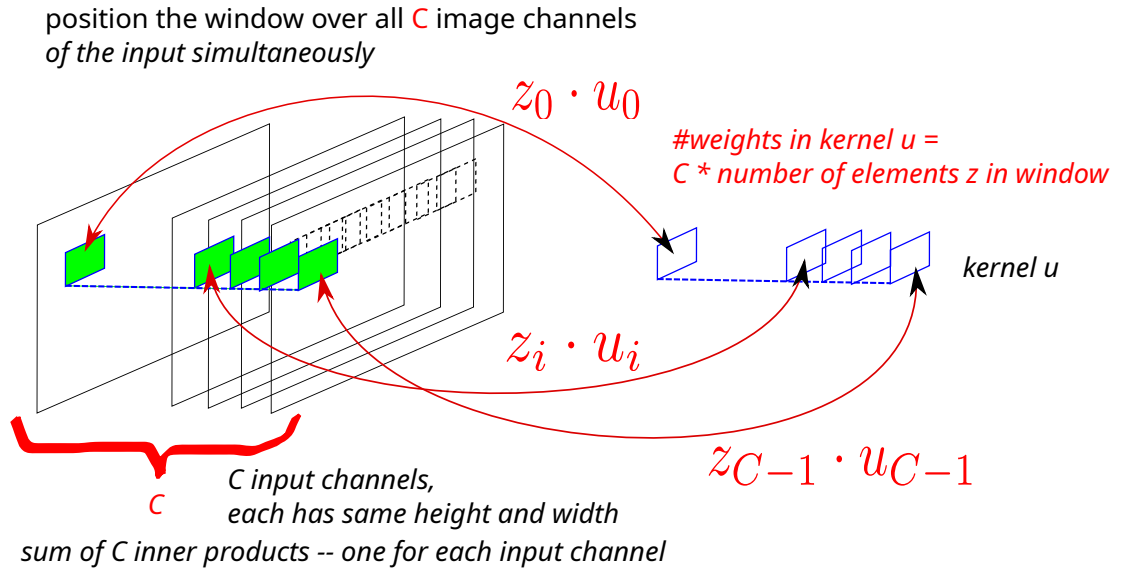
The window is sliding, for 1D-convolutions along 1 dimensions, for 2D-convolutions along 2 dimensions.

2.6 dealing with multiple input channels

The difference when changing to multiple input channels:

- The kernel has the same number of channels as the input.
- The sliding window moves synchronized for all channels
- One computes one inner product for each input channel, and adds the result up for all channels.

A 2D-convolution with multiple input channels and one output channel can be drawn like this:



$$r[h, w] = z_0[h : h + k_h, w : w + k_w] \cdot u_0 + \dots + z_i[h : h + k_h, w : w + k_w] \cdot u_i + \dots + z_{C-1}[h : h + k_h, w : w + k_w] \cdot u_{C-1}$$

Above shows convolution for a convolution kernel with 1 output channel. Finally, one convolution layer usually uses O independent convolutional kernels, resulting in O channels as output.

In math, one output element of a 2D-convolution with C input channels and 1 output channel can be written as:

$$\begin{aligned} z[h, w] &= \sum_{i_h=0}^{k_h} \sum_{i_w=0}^{k_w} \sum_{c=1}^C u[c, i_h, i_w] x[c, h + i_h, w + i_w] \\ &= u \cdot x[:, h : h + k_h, w : w + k_w] \end{aligned}$$

This is still an inner product between $u[:, :, :]$ and the slice $x[:, h : h + k_h, w : w + k_w]$ of tensor x .

The convolution kernel has one additional dimension, besides the number of dimension used to run the sliding window.

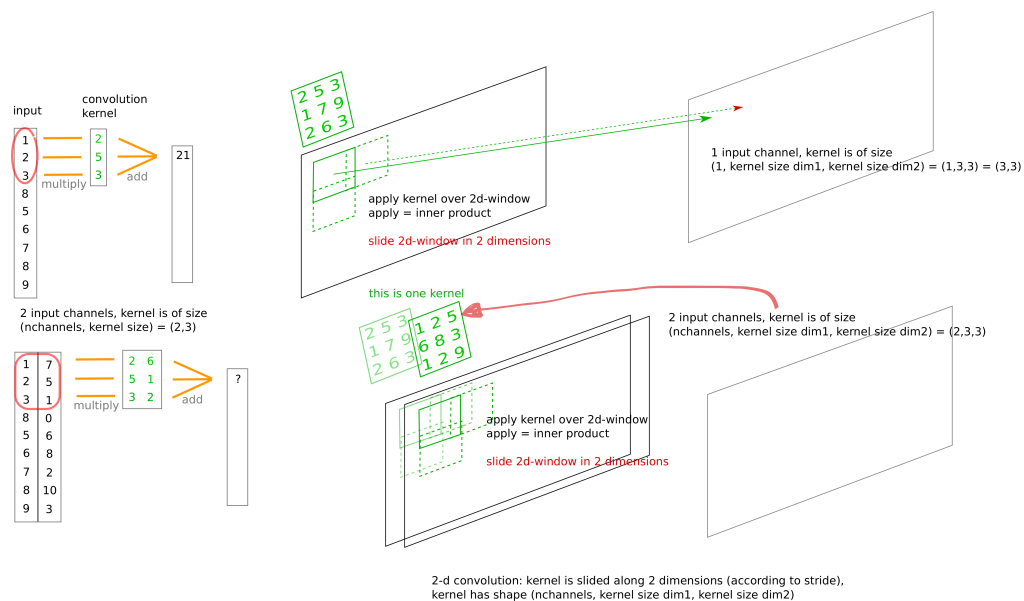
This is to process multiple input channels. In this additional it has as many entries as the number of input channels in the feature map to be processed.

In math, one output element of a 2D-convolution with C input channels and D output channels can be written as:

$$z[d, h, w] = \sum_{i_h=0}^{k_h-1} \sum_{i_w=0}^{k_w-1} \sum_{c=0}^{C-1} u[c, i_h, i_w, d] x[c, h + i_h, w + i_w]$$

$$= u[:, :, :, d] \cdot x[:, h : h + k_h, w : w + k_w], \quad d \in 0, \dots, D - 1$$

compare 1d vs 2d convolutions:



- a convolution at one fixed region (region means here: rectangular window) x of an image is an inner product $w \cdot x$ between kernel weights u and this region x - this is a single real number.

- We slide the convolutional kernel u over the image/input tensor and compute the inner product over windows of the input tensor located at different positions.

In convolution we apply the same kernel u across all locations for computing inner products.

- the values of the kernel u are the parameters learned during training!

2.7 A small exercise I

Computing the number of trainable parameters.

For a d-dimensional standard convolution (no groups, no depth-wise property) with kernel size $(k_0, k_1, \dots, k_{d-1})$, c_i input channels and c_o output channels and bias terms we have:

$$\left(\prod_{l=0}^{d-1} k_l \right) c_i c_o + c_o$$

This is important to understand the spatial resolution of feature maps for a given input image.

- (a) a 1d-convolution with kernel size (5), 3 input channels, 4 output channels, no bias
- (b) a 2d-convolution with kernel size (8,8), 10 input channels, 5 output channels, with bias
- (c) a 2d-convolution with kernel size (2,4), 5 input channels, 2 output channels, with bias
- (d) a 3d-convolution with kernel size (2,2,3), 2 input channels, 2 output channels, no bias

2.8 why use convolutions, argument II – a convolution layer as a battery of pattern detectors

Convolution implements a battery of localized pattern detectors all over the input tensor.

Why is that useful?

an object ...



can appear anywhere in an image/input tensor



Therefore one wants to be able to perform a detection over all positions in an input tensor.

In order to see why:

- kernel matrix w is some pattern (Krizhevsky paper, Zeiler paper)
- apply convolution, get $y = w \cdot inputwindow + b$

inner product is a similarity measure, high positive for inputs parallel to kernel, zero for inputs orthogonal, high negative for inputs antiparallel to kernel

- apply convolution with activation function $g(\cdot)$ in the next layer – result for one window has formula $g(kernel \cdot inputwindow + b)$, detector for patterns in input channels similar to the kernel
- detection is performed all over across the sliding space (1-d,2-d,3-d,n-d)
- compare to fully connected layer: detection is global for fully connected, localized for convolutions

Convolution allows to perform a pattern detection in a translation-invariant manner!

2.9 Kernel parameters and their influence on the output shape

What is the size of the output matrix ? Suppose we use 2d convolutions and inputs are $M \times N$. First observation: analysis can be done separately for every

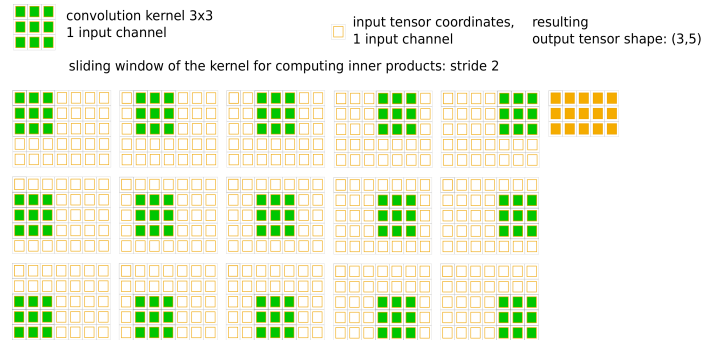
sliding dimension.

Takeaway: there are three parameters influencing the output size: padding, kernel size, and stride.

It is clear what kernel size is – the shape of w .

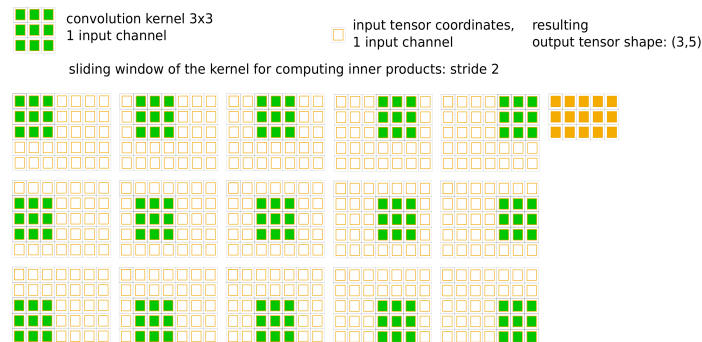
shape without padding:

Problem: if we apply a kernel to a window, it must fit into it. An array of length M starts at 0 and ends at $M - 1$.



If we use a 3×3 convolution kernel w , if its left border start at index 0, then its right border ends at index 2. Thus the last start can be only at index $M - 3$, because then its right border would be at $M - 3 + 2 = M - 1$. $\{0, 1, \dots, M - 3\}$ are $M - 2$ elements.

Result is: if we move w always by one pixel (=stride 1), then the output is $(M - 2) \times (N - 2)$.



For a 5×5 kernel: if its left border start at index 0, then its right border ends at index 4. Thus the last start can be only at index $M - 5$, totalling $M - 4$ elements

In general without padding, if we apply a kernel of size $k \times k$, and if we move w always by a stride of 1, then the output shape will be $(M - k + 1) \times (N - k + 1)$

What is if we use a stride s larger than one ?

stride is the number of input elements/pixels which w is moved in every step

draw a stride fig / ksize 3,5

For a $k \times k$ kernel we start with the left boundary at 0,
 move always $0, 0 + s, 0 + 2s, 0 + 3s$
 and end at the largest index c such that $cs + k - 1 \leq M - 1$ holds.

How many sliding windows do we have as a function of input size M
 ($M = k$ means image is as large as the kernel size k)?

$M = k$	$\mapsto 1, c = 0$
$M = k + s$	$\mapsto 2, c = 1$
$M = k + 2s$	$\mapsto 3, c = 2$
$M = k + 3s$	$\mapsto 4, c = 3$
$M = k + 4s$	$\mapsto 5, c = 4$

Therefore: we seek the largest c such that

$$cs + k - 1 \leq M - 1$$

this means:

$$c \leq \frac{M - k}{s}$$

However c is an integer, $\frac{M-k}{s}$ might be a fractional number. consider examples

- $\frac{M-k}{s} = 2.4$ – then the largest $c = 2$
- $\frac{M-k}{s} = 3.3$ – then the largest $c = 3$
- $\frac{M-k}{s} = 5.9$ – then the largest $c = 5$
- $\frac{M-k}{s} = 5.0$ – then the largest $c = 5$
- $\frac{M-k}{s} = 3.0$ – then the largest $c = 3$

in all cases $c = \text{floor}(\frac{M-k}{s})$ ($\text{floor}(v)$ is the largest integer $\leq v$), therefore

$$c \leq \text{floor}(\frac{M-k}{s})$$

As seen above, we have always $c + 1$ outputs, therefore the output size is

$$\text{floor}(\frac{M-k}{s}) + 1 = \text{floor}(\frac{M-k}{s} + 1)$$

If we have an input of length M in one dimension, then the output size in that dimension for a kernel of size $ksize$ and stride s without padding is given as: $\text{floor}((M - ksize)/s + 1)$

Example: 5×7 , $ksize = (3 \times 1)$, $s = 2$, then: output is $\text{floor}(\frac{5-3}{2} + 1) \times \text{floor}(\frac{7-1}{2} + 1) = 2 \times 4$

shape with padding:

Padding means to add for every dimension at both ends of an input a layer of zeros. Example: 2d-input, pad 2 means:

0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0							0	0
0	0							0	0
0	0							0	0
0	0							0	0
0	0							0	0
0	0							0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

Padding of 2

- add two columns at the beginning
- add two columns at the end
- add two rows at the beginning
- add two rows at the end
- $M \times N \rightarrow (M + 4) \times (N + 4)$

In general: padding by r changes the input shape $M \times N \rightarrow (M + 2r) \times (N + 2r)$

If we pad by r , then the image dimension increases from M to $M + 2r$, therefore:

the output kernel size for a kernel of size $ksize$ and stride s with padding of r is given as: $\text{floor}((M + 2r - ksize)/s + 1)$

Standard padding: Standard padding is used if we pad for a kernel of size $ksize = 2r + 1$ by a pad value r .
In such a case the output shape is $\text{floor}((M - 1)/s + 1)$ – independent of the kernel size (padding is adaptive).

Observations:

- stride s shrinks an output shape much more than kernel size $ksize$ does ... see $\text{floor}((M - ksize)/s + 1)$
- if we use standard padding (which is adaptive), then kernel size has no influence on the output shape
- Qualitative impact of stride: Convolution with stride s takes an image with height (h, w) and creates a downsampled image with dimensions being approximately $(h/s, w/s)$

https://github.com/vdumoulin/conv_arithmetic

2d-Convolution as recapitulation:

A 2d-convolutional layer applies a convolutional kernel w over a multi-channel image (that is an 3-dimensional array having format $C \times width \times height$). Application means here: the kernel is slid along 2-dimensions (height,width) of the multi-channel image according to a stride. Everytime it stops over a rectangular window, an inner product between that kernel and the rectangular window is computed. This inner product is a real number. By sliding the kernel, one obtains as output one matrix per kernel. Using multiple kernels results in a multi-channel image with format $\#kernels \times newwidth \times newheight$. The weights w for all the kernels are learnt during neural network training.

In convolution we apply one weight vector w over many positions in one set of input channels – why sharing a w across the image makes sense ?
the inner product between a window of the input layer and the weight w can be seen as using w as a “detector”.

- One wants to learn the detector over all regions in the image.
- when one has found a good detector, one wants to apply the same detector over all regions in the image
- for these reasons w is shared across the image.
- convolutional neural nets: sliding window of inner products, combines **neighboring/localized neurons** into a neuron in the next layer
- original paper: *Neocognitron: A self-organizing neural network model for a mechanism of pattern recognition unaffected by shift in position.* K. Fukushima, Biological Cybernetics, 1980

2.10 A small exercise II

Compute the output feature map shape (assume batch-size is 1)

- (a) input shape $(c, t) = (5, 20)$, 1d-conv, kernel size (6), 12 output channels, stride 2, padding 1
- (b) input shape $(c, h, w) = (4, 32, 32)$, 2d-conv, kernel size (5, 5), 16 output channels, stride 3, padding 2
- (c) input shape $(c, h, w) = (12, 64, 64)$, 2d-conv, kernel size (7, 5), 3 output channels, stride (2, 1), padding (2, 2)

2.11 The theoretical receptive field

- 3×3 , stride= 1 kernel has a field of view of 3
- if we stack two kernels 3×3 , stride= 1 – field of view ? 5!
- if we stack three kernels 3×3 , stride= 1 – field of view ? 7!
- 3×3 , stride= 1 kernel has a field of view of 3
- if we stack two kernels 3×3 , stride= 2 and 3×3 , any stride – field of view ? 7!

Important: the stride of the first kernel plays a role for the second feature map, the stride of the second kernel would be of importance only for the third feature map!

Let R be the receptive field size at layer $k \geq 1$. S_k the stride in layer k , F_k the kernel size in layer k .
Then the theoretical receptive field will be:

$$R_k = R_{k-1} + (F_k - 1) \prod_{i=1}^{k-1} S_i$$

What is preferable to use?

- one 5×5 , stride= 1 or a stack of two 3×3 , stride= 1 ??
- both have the same receptive size

2.12 why use convolutions, argument III? – stacking convolutions

- Think of the image in the above graphic not as an image, but as a grid showing signals of detectors.

An RGB-input image itself is a signal of detectors (camera sensors)

If the input tensor is the output of a previous convolution layer with kernel v , then the input is a detector for the signal encoded by v , because it measures similarity to v in every spatial location.

Then every input “pixel” of the input feature map is a signal of some detector for a kind of part/structure, e.g. v may encode an eye, a furry ear or a car wheel.

A convolution is a weighted sum of inputs

$$z[d, h, w] = \sum_{i_h=0}^{k_h-1} \sum_{i_w=0}^{k_w-1} \sum_{c=0}^{C-1} u[c, i_h, i_w, d] x[c, h + i_h, w + i_w]$$

$\Rightarrow$ It is a weighted sum of signals of different detectors (by it sum over different channels c) over neighboring spatial positions (sum over i, j).

A convolution allows to learn a combination of part detectors that are spatially *neighboring*.

- Why it is ok to learn a neighboring combination of parts? Semantically meaningfully parts in an image (eye, fur, leg, whole cat) form usually a connected, neighboring region in an image ¹. So **a convolution at one fixed output point learns to combine neighboring parts**.
- another thought: by stacking convolution layers one can look at regions that get larger with every layer:
- If one stacks two such layers by 3×3 kernels, then the first layer looks at 3×3 , but the next layer looks at 5×5 regions (and the third 7×7). So each layer looks at regions in the input image with a larger size.

Neurons at high/late layers look at very large regions of the input image

- convolutions can be applied as 1d-convolution to any sequence data, like time series, language sentences, DNA sequences.

The philosophy:

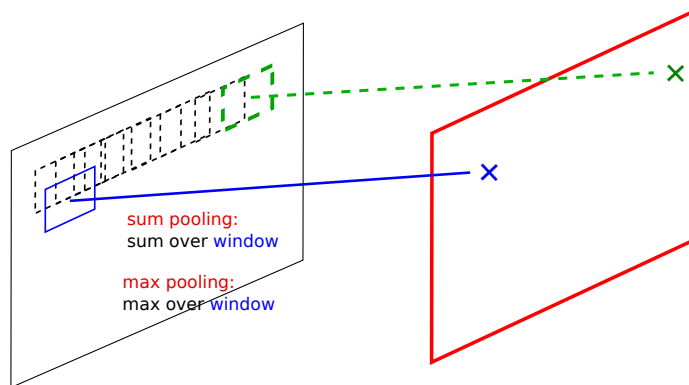
- machine learning in general: keep number of parameters limited relative to training set size
- deep learning in particular: better stack simple functions in many layers than trying to learn something very complex in one layer.

¹What would be a counterexample?

3 Pooling

Its related to convolution, but has no parameters to be learned. In case of multi channels: each channel separately treated, usually not combined.

- **Sum pooling:** Sums up all the elements over a window and returns a real number. Same sliding window approach with kernel size (= window size) and stride – yields then a matrix as output.



- **Average pooling:** does not sum, but computes the average over the number of elements
- Equivalent to a Filter kernel $w = \text{const}$
- same as convolution: works with M input channels – but usually each channel is pooled separately!
- Idea: Often after convolution+activation ... average of detector outputs
- **Max pooling:** Computes a max up all the elements over a window and returns a real number. Same sliding window approach with kernel size (= window size) and stride – yields then a matrix as output.
- Equivalent to a Filter kernel $w = \text{const}$ and replace aggregation: sum gets replaced by a max (see that you could plug in other aggregation operations).
- Idea: Often after convolution+activation ... highest detector output over a field of view

4 Old-style (2014 ... before-batchnorm and before residual connections) Neural Network structure for Computer vision tasks

- **Convolution-Relu-Pooling** Repeated. Last 1 – 2 layers are a **fully connected** layer.

- ReLU: Rectified linear $g(x) = \max(x, 0)$.
- typical examples: LeNet, VGG-16

note: Nowadays more alternatives for activation functions: leaky ReLU, eLU, SeLU, GeLU, Swish. Sigmoids are used for bounded regression outputs and for attention weights, not common for vision directly after a convolution.

5 Special Designs

5.1 Dilated convolutions / a-trous convolutions / atrous convolutions

atrous - its French

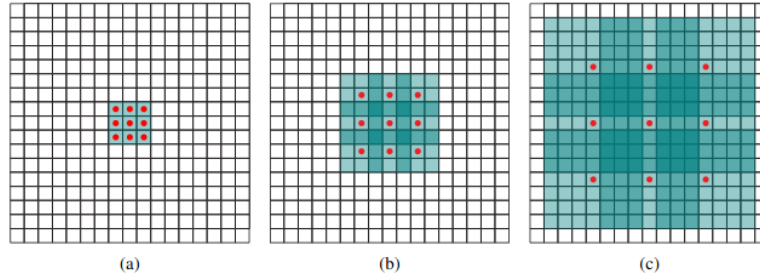


Figure 1: Systematic dilation supports exponential expansion of the receptive field without loss of resolution or coverage. (a) F_1 is produced from F_0 by a 1-dilated convolution; each element in F_1 has a receptive field of 3×3 . (b) F_2 is produced from F_1 by a 2-dilated convolution; each element in F_2 has a receptive field of 7×7 . (c) F_3 is produced from F_2 by a 4-dilated convolution; each element in F_3 has a receptive field of 15×15 . The number of parameters associated with each layer is identical. The receptive field grows exponentially while the number of parameters grows linearly.

source: <https://arxiv.org/pdf/1511.07122.pdf>

- dilation factor $d = 1$ - usual convolution
- dilation factor $d = 2$ - put between each two elements in u 1 zero, apply convolution with zeros-expanded u_* , effectively uses every second value of the input feature map
- dilation factor $d = 4$ - put between each two elements in u 3 zeros, apply convolution with zeros-expanded u_* , effectively uses every fourth value of the input feature map
- putting $d - 1$ zeros into the kernel – can be seen as skipping $d - 1$ elements in the input each time, and using only each d -th element in the input:

$$z[h] = \sum_{i_h=0}^{k-1} w[i_h] x[h + i_h * d]$$

https://github.com/vdumoulin/conv_arithmetic

- idea: obtain larger receptive fields, looks at larger contexts
- used in segmentation methods e.g. deeplab v2/v3 <https://arxiv.org/abs/1606.00915>, <https://arxiv.org/abs/1706.05587>

5.2 fractionally strided convolutions

Not in this lecture. For curiosity: One input element will be multiplied element-wise with each kernel element. The result is not a real number but has the shape of the kernel. The result will be added to the output.

5.3 Depth-wise separable convolutions

This is a pair of two convolution layers (with activation functions)

- Separable convolution with a kernel of size $k \times k$ (for 2D-convolutions)
- 1×1 convolution

For a good overview: <https://www.geeksforgeeks.org/depth-wise-separable-convolutional-neural-networks/>.

Separable convolution:

- Restriction 1: Same number of output channels as the number of input channels.
- Let the kernel W be of shape $(C, 1, h, w)$ where h, w is the spatial kernel size.
- Restriction 2: the i -th input channel is convolved with $W[i, 0, :, :]$ to produce the i -th output channel.
 - Difference to vanilla convolution: each input channel is processed separately. Information is **not** combined across input channels. Information is only combined across spatial neighbors within each channel.
- realized in pytorch by setting the number of groups to be equal to the number of input and output channels.

The subsequent 1×1 convolution combines different channels (for the same spatial coordinate $[h, w]$).

Why used?

- Depth-wise separable convolutions has a lower number of trainable parameters compared to a single standard convolution (but also more restricted in what patterns it can detect)
- depth-wise $+1 \times 1$ conv has $hwC + C^2$ parameters
- vanilla convolution with $C_{in} = C_{out} = C$ has hwC^2 parameters
- for $h = w = 3$ it is: $9C + C^2$ vs $9C^2$. For $C \geq 2$ this is more efficient